

The Iterative Loop of ML Development

Developing a machine learning model is an **iterative process**. It is very rare for the first version of a model to work as well as you want. The key to building an effective system is to efficiently cycle through the development loop.

The process is as follows:

1. **Choose Architecture:** Start by making decisions about your system. This includes selecting the model (e.g., neural network architecture), choosing the data to use, and setting initial hyperparameters.
 2. **Train Model:** Implement and train the chosen model on your training data.
 3. **Run Diagnostics:** The initial model will almost never be perfect. You must run diagnostics to gain insight into *why* it's not working well. Key diagnostics include **bias/variance analysis** and **error analysis**.
 4. **Make Decisions & Iterate:** Use the insights from your diagnostics to decide on the most promising next steps (e.g., "the model has high bias, so I need to add more features"). Then, return to step 1 and repeat the loop.
-

Case Study: Email Spam Classifier

This iterative process can be illustrated with a **text classification** problem, such as building a spam filter.

- **Input (x):** Features of an email.
- **Output (y):** 1 for Spam, 0 for Not-Spam.

Initial Feature Engineering

A common way to create features for a text document is to:

1. Define a **dictionary** of the top 10,000 (or more) most common words.
 2. Represent the email as a **10,000-element feature vector**.
 3. Set $x_j = 1$ if the j -th word from the dictionary appears in the email, and $x_j = 0$ if it does not.
 4. (Alternatively, x_j could be the *count* of how many times the j -th word appears).
 5. Train a logistic regression or neural network on this input vector x .
-

The "What to Try Next?" Problem

If your initial spam classifier performs poorly, you will have many ideas for how to improve it. The challenge is choosing the *right* one, as some options are very time-consuming.

Possible Ideas:

- **Collect more data:** (e.g., start a "honeypot project" by creating fake email addresses to attract spam).
 - **Develop better features (Routing):** Analyze the email **header** (routing information) to see if it came from a known spam server.
 - **Develop better features (Body):**
 - Combine different forms of a word (e.g., treat "discount" and "discounting" as the same).
 - Create an algorithm to detect deliberate **misspellings** (e.g., "w4tches", "mortgages").
-

The Role of Diagnostics

Instead of guessing, you should use **diagnostics** to decide which path is most promising.

- **Bias/variance analysis** can tell you *if* collecting more data is a good use of time.
- If the model has **high variance** (overfitting), then the honeypot project (getting more data) is a very good idea.
- If the model has **high bias** (underfitting), then getting more data will *not* help, and you should instead focus your time on engineering better features (like the misspelling detector).

The next key diagnostic to learn is **error analysis**.